PROJECT DOCUMENTATION

RAVI System

Recognition Automated Via Infrared

Onboarding guide for the FTIR microplastic identification platform — covering the Flutter client, the Python MLP inference server, and the Firebase realtime database schema. Aimed at undergraduate research assistants joining the project for the first time.

Project	RAVI — Microplastic Identification by FTIR
Author	Daniel Martins (PhD candidate)
Audience	Lucas — Undergraduate Research (IC)
Components	Flutter app · FastAPI server · Firebase RTDB
Repository	github.com/<org>/ravi-system `(replace with the real URL)`
Stack	Dart 3.11 · Python 3.13 · Keras/TensorFlow · FastAPI

This document is generated from build_docs.py — re-run after editing to keep it in sync.

SECTION 00

Table of Contents

SECTION 01

System overview

RAVI is a desktop and web application that supports microplastic research workflows: cataloguing field collections, registering individual samples, importing FTIR spectral data, running automatic polymer identification through a multilayer perceptron (MLP) model, and visualising the spectrum together with the model's attention map.

The system is `multi-tenant`: each institution / laboratory has its own isolated namespace inside Firebase. Users from one institution cannot see or modify data from another. Only administrators can register new institutions, which prevents accidental tenant pollution during early adoption.

What it solves

- Centralises FTIR collections across collaborators in a single structured database, replacing scattered spreadsheets.
- Automates polymer identification (PE, PP, PS, PA, EVA, cellulose) by reusing the trained MLP from the BRACIS 2026 paper pipeline.
- Stores the spectral signal and the model's saliency map together, so reviewers can visually justify each identification.
- Provides traceable random sample IDs that do not leak collection order or operator-chosen names from the original CSV.

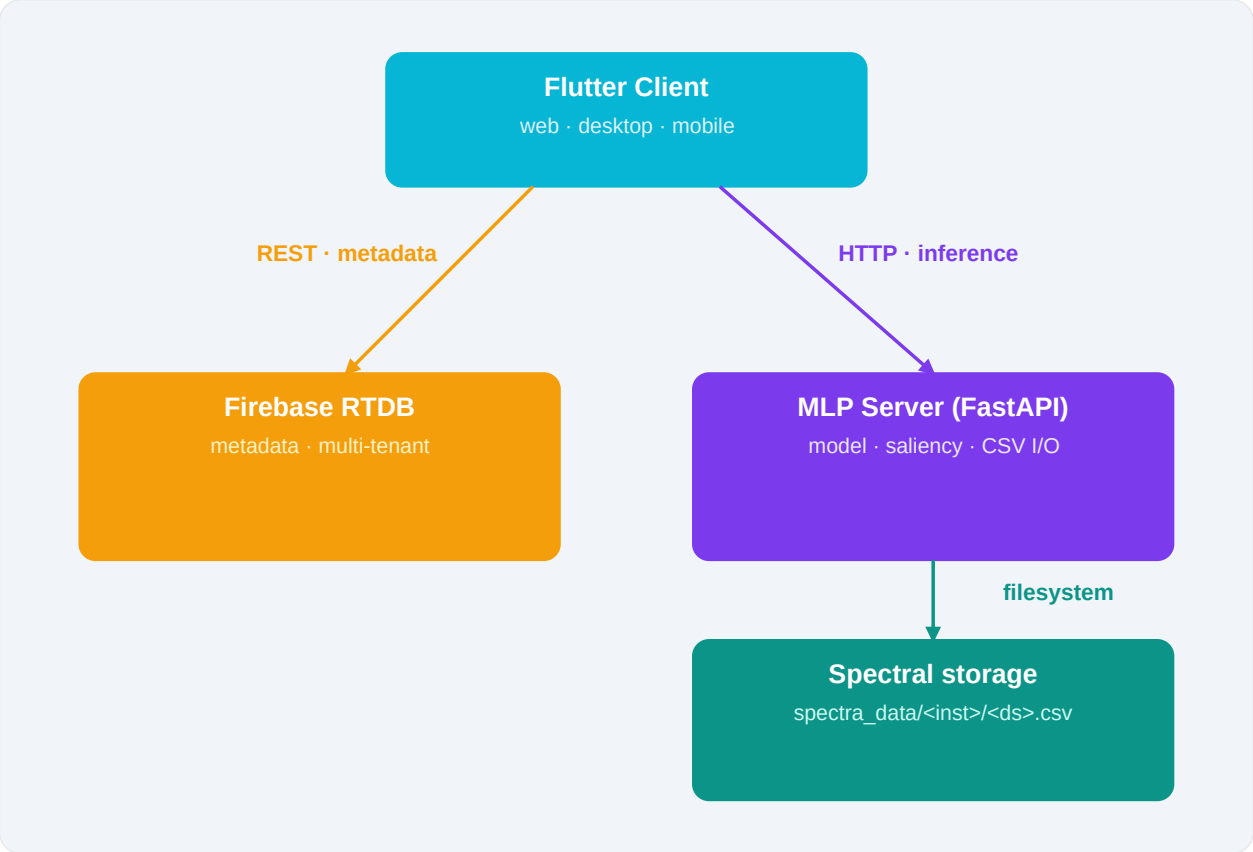
Out of scope (for now)

- Production-grade authentication: passwords are obfuscated with a salted base64 hash, not bcrypt/argon2. Plan: migrate to Firebase Auth before any external rollout.
- Granular per-collection permissions: every user inside an institution sees every collection of that institution.
- Automated retraining loop: verified samples are stored, but the training script is run manually outside the app.

SECTION 02

Architecture and component map

Three independent processes cooperate. The Flutter client is the only thing the end user sees. It talks to two backends over HTTP: Firebase Realtime Database (metadata) and the local FastAPI server (model inference + bulky spectral data).



Why two backends?

Metadata (sample names, polymer labels, attention summaries) lives in Firebase: small, indexed, multi-user. The raw spectrum (~1763 intensities per sample) would bloat each Firebase node, so it is stored as plain CSV on the inference server, keyed by sample ID. When the detail screen opens, the client fetches metadata from Firebase and the spectrum from the server, then renders both in the same chart.

Default network endpoints

SERVICE	URL	STARTED BY
Flutter dev server	`http://localhost:<port>`	`flutter run` (port chosen by Flutter)
MLP server	`http://localhost:8000`	`python mlp_server.py`

Firebase RTDB	`https://<project>.firebaseio.com`	Hardcoded in `firebase_service.dart`

SECTION 03

Repository layout

Every meaningful directory and file is listed below. Directories are highlighted in cyan; the rest are source files. Two areas deserve special attention: `lib/services/` is the only place that opens HTTP connections, and `lib/l10n/` holds the source-of-truth translations — every other file in that directory is generated automatically by `flutter gen-l10n`.

PATH	DESCRIPTION
■ **Sistema/**	Repository root.
`mlp_server.py`	FastAPI inference + spectra storage.
`extract_scaler.py`	Builds scaler_params.json from training data.
`MLP_best_model.h5`	Trained Keras model (FTIR → polymer).
`scaler_params.json`	MinMaxScaler params used by the server.
`DB_10cl_100sp.csv`	Sample CSV used during development.
`TDB.csv`	Test database.
■ **spectra_data/**	Created at runtime — per-institution CSVs.
`/.csv`	One file per dataset, one row per sample.
■ **docs/**	Output directory for this PDF.
■ **deepmicroplastic/**	Flutter project root.
■ **lib/**	Dart source code.
`main.dart`	App entry; wires the locale service.
■ **l10n/**	Translations (ARB) + generated Dart.
`app_en.arb`	English translations (default).
`app_pt.arb`	Portuguese translations.
■ **models/**	Plain Dart data classes.
`institution_model.dart`	InstitutionModel + slugify().
`user_model.dart`	UserModel with institutionSlug.
`spectrum_model.dart`	Polymer enum, sample, dataset.
■ **services/**	All HTTP calls live here.

`firebase_service.dart`	Raw RTDB REST wrapper.
`institution_service.dart`	Institution CRUD.
`auth_service.dart`	Login + user CRUD per tenant.
`dataset_service.dart`	Dataset CRUD per tenant.
`sample_service.dart`	Sample CRUD + spectrum hydration.
`spectral_storage_service.dart`	Talks to /spectra endpoints.
`csv_import_service.dart`	Bulk MLP via /predict_csv.
`mlp_service.dart`	Single-sample MLP via /predict.
`locale_service.dart`	Persists EN/PT choice.
■ **screens/**	One screen per .dart file.
`login_screen.dart`	3-stage login flow + image bg.
`new_institution_screen.dart`	Create tenant + initial admin.
`ftir_overview_screen.dart`	Home screen (post-login).
`manage_institutions_screen.dart`	Admin-only tenant list.
`manage_users_screen.dart`	Admin-only user list per tenant.
`user_form_screen.dart`	Create / edit user.
`add_dataset_screen.dart`	Register a new collection.
`dataset_detail_screen.dart`	Sample list of a collection.
`add_sample_screen.dart`	Single, batch, or CSV-attached.
`sample_detail_screen.dart`	Spectrum + identification view.
`import_csv_screen.dart`	Bulk CSV import.
■ **utils/**	Pure helper functions.
`id_generator.dart`	Random traceable IDs.
■ **widgets/**	Reusable UI components.
`ftir_chart.dart`	Custom-painted FTIR plot.
`language_menu.dart`	EN/PT dropdown.
■ **assets/**	Image assets.

`login_bg.jpg`	Faded background on login.
`pubspec.yaml`	Flutter manifest + deps.
`l10n.yaml`	Localisation generator config.

SECTION 04

Setup and running locally

Prerequisites

- Python 3.13+ — verify with `python3 --version`.
- Flutter 3.11+ on the stable channel — verify with `flutter --version`.
- A Firebase Realtime Database project. The default URL is hardcoded in `lib/services/firebase_service.dart`; change it before deploying outside the lab.
- A Linux, macOS or Windows workstation with at least 4 GB free RAM (TensorFlow load + Flutter dev tools).

Step 1 — Clone the repository

Pick the directory where you keep your projects and clone there. Anywhere works — the rest of the documentation refers to that path as `$RAVI_HOME`.

```
# Replace <REPO_URL> by the actual git URL the team uses.
# Pick any folder you like — the path is referenced as $RAVI_HOME below.
git clone <REPO_URL> ~/projects/ravi-system
cd ~/projects/ravi-system

# (Optional) Make the path easy to refer to in your shell:
echo 'export RAVI_HOME="$HOME/projects/ravi-system"' >> ~/.bashrc
source ~/.bashrc
```

Step 2 — Start the MLP server

The server uses a virtual environment isolated from the system Python. Activate it once and keep the terminal open while you work.

```
# All Python deps live in a venv to avoid polluting your system Python.
cd "$RAVI_HOME"
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn tensorflow pandas numpy pydantic

# Start the server (keep this terminal open):
python mlp_server.py
# → Loading model from MLP_best_model.h5 ...
# → Uvicorn running on http://0.0.0.0:8000
```

Step 3 — Start the Flutter client

In a second terminal, install Dart dependencies and launch the app. On first run, choose `-d chrome` — the web target is the easiest to debug.

```
# In a second terminal:
cd "$RAVI_HOME/deepmicroplastic"
flutter pub get
flutter gen-l10n            # only after editing app_*.arb
```

```
flutter run -d chrome       # or -d linux / -d macos / -d windows
```

Step 4 — Optional: sandbox configuration

If you use Claude Code or another IDE assistant with sandboxed file access, add the project to its allowed paths. This avoids permission prompts each time the assistant tries to read or edit a file.

```
# Tell Claude Code (or any IDE assistant) where the project lives, so its
# sandbox can read/write inside it without extra prompts:
claude --add-dir "$RAVI_HOME"

# Or, for a long-lived setting, edit ~/.claude/settings.json:
#   "permissions": {
#     "additionalDirectories": ["$HOME/projects/ravi-system"]
#   }
```

First-run experience

If the Firebase database has no `/institutions` node, the login screen automatically opens the institution-creation flow. You fill in the institution name, an admin username and password — after saving, the app boots straight into the home screen as that admin. From there, additional institutions can be added through *Menu → Manage Institutions*.

SECTION 05

MLP inference server

FastAPI process loaded once at startup. It owns the trained model (`MLP_best_model.h5`), the optional scaler (`scaler_params.json`), and the on-disk CSVs that store raw spectra per dataset. CORS is wide-open — fine for the lab network, tighten before any public deployment.

Endpoints

METHOD	PATH	BODY / PARAMS	PURPOSE
GET	`/health`	—	Returns scaler mode + class list
POST	`/predict`	`{wavenumbers, intensities}`	Single-sample inference
POST	`/predict_csv`	multipart file	Batch inference; one CSV row per sample
POST	`/spectra/{inst}/{ds}`	`{sample_id, wavenumbers, intensities}`	Persist a sample spectrum to disk
GET	`/spectra/{inst}/{ds}/{sample_id}`	—	Reload that spectrum
DELETE	`/spectra/{inst}/{ds}/{sample_id}`	—	Remove a single row from the dataset CSV

Inference pipeline

For each sample the server: (1) interpolates the user's (wavenumber, intensity) pairs onto the model's fixed grid of 1763 points from 3998 to 600 cm■¹; (2) applies the MinMaxScaler if `scaler_params.json` is present, otherwise falls back to per-sample normalisation; (3) runs the Keras model and computes the saliency by `|grad × x|`, normalised to [0, 1]. The response carries the predicted class, the confidence, the wavenumbers / intensities and the saliency vector aligned with the wavenumber grid.

Polymer classes

The trained model recognises six classes: `EVA, PA, PE, PP, PS, cellulose_like`. The Flutter client maps these to its `PolymerType` enum and renders them with consistent colours and full names (see `lib/models/spectrum_model.dart`).

SECTION 06

Spectral storage (CSV files)

Each dataset has its own CSV file under `spectra_data/<institution_slug>/<dataset_id>.csv`. The first column holds the random sample ID, the rest are the intensities indexed by the wavenumbers (CSV header). When the detail screen opens, the Flutter client requests the row by ID and rebuilds the spectrum on the fly.

```
sample_id,3998.0,3995.7,3993.4, ... ,602.4,600.0
smp-x9p7k2qa3jvw,0.0123,0.0119,0.0115, ... ,0.0341,0.0322
smp-3kk2pqahjabc,0.0421,0.0414,0.0407, ... ,0.0091,0.0088
```

Why a CSV per dataset?

- Firebase RTDB has practical limits on node size — embedding 1763 floats per sample would slow every list query.
- Plain CSV is exportable for retraining without ETL: each file is already a valid input for the training pipeline.
- Tenant separation is enforced at the filesystem level: the directory layout makes it impossible to read another institution's spectrum by accident.

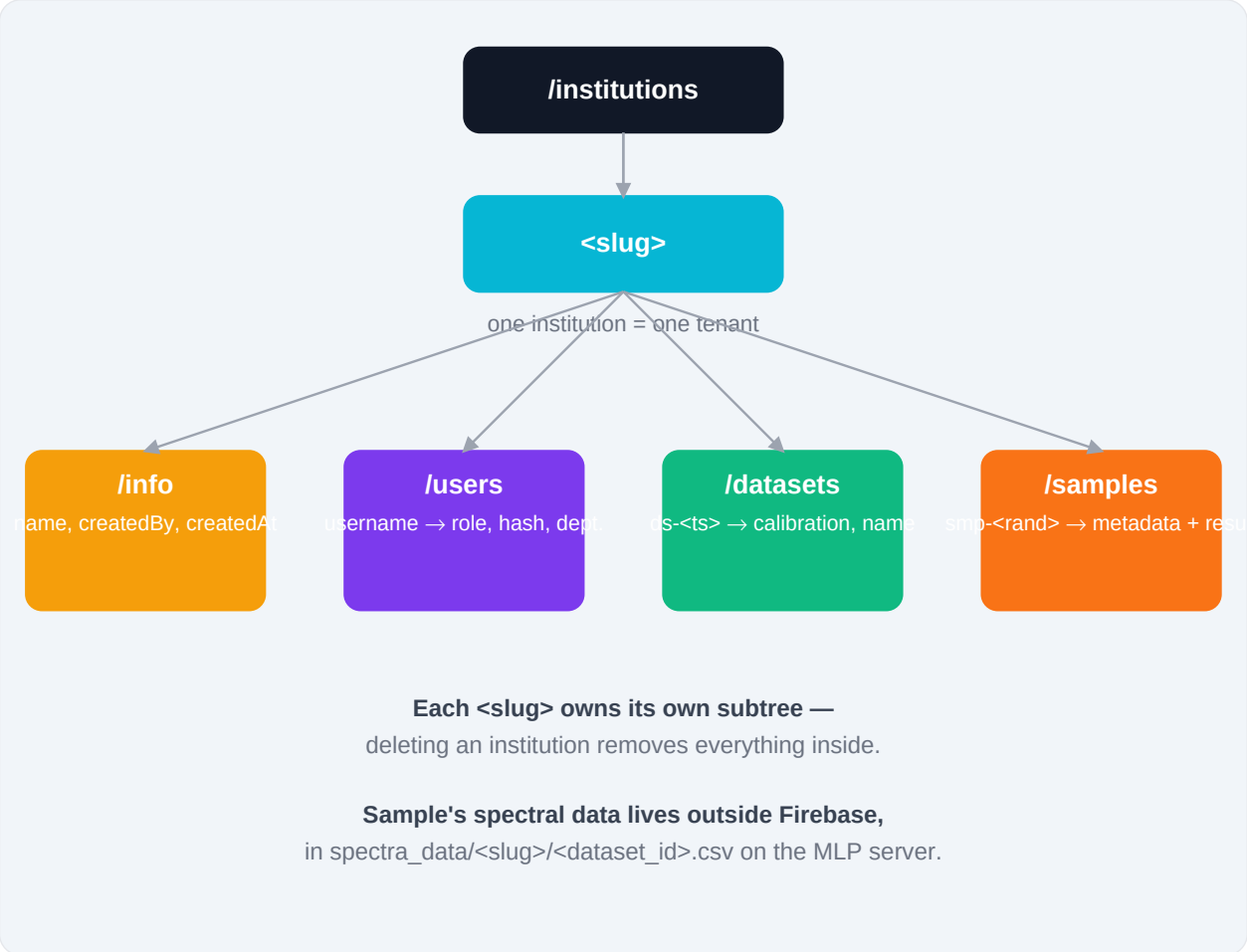
Re-aligning headers

If a sample comes from a CSV with a slightly different wavenumber grid (e.g. 1700 columns instead of 1763), `save_spectrum` interpolates the new intensities onto the dataset's existing header before appending. This guarantees the file stays rectangular and importable into pandas.

SECTION 07

Firebase database schema

Every entity lives under `/institutions/<slug>`, so removing a tenant is a single recursive delete. The Flutter services already enforce this layout — no screen ever builds a raw Firebase path.



Each <slug> owns its own subtree —
deleting an institution removes everything inside.

Sample's spectral data lives outside Firebase,
in spectra_data/<slug>/<dataset_id>.csv on the MLP server.

/info — institution metadata

FIELD	TYPE	EXAMPLE
`name`	string	USP — IAG
`createdBy`	string	admin
`createdAt`	int (ms epoch)	1714588800000

/users/{username} — credentials and profile

FIELD	TYPE	EXAMPLE

`name`	string	João Silva
`email`	string	jsilva@usp.br
`department`	string	Lab. Oceanografia
`role`	string	admin · researcher
`passwordHash`	string	base64('raviDeepMp_' + pwd)
`createdAt`	int	1714588800000

/datasets/{datasetId} — collection metadata

FIELD	TYPE	EXAMPLE
`name`	string	Praia do Futuro — Apr/2024
`description`	string	Surface sediment, 5 points
`location`	string	Fortaleza, CE
`createdBy`	string	jsilva
`microscopeMode`	enum	atr · transmission · reflection
`microscopeModel`	string	Bruker Vertex 70 + Hyperion 3000
`resolution`	number	4.0
`numScans`	int	64
`detectorType`	string	MCT
`crystalType`	string	Diamante
`dataType`	enum	absorbance · transmittance

/samples/{sampleId} — individual sample (no spectrum)

FIELD	TYPE	EXAMPLE
`datasetId`	string (FK)	ds-1714588900000
`name`	string	PF-K7XN3F (display name)
`collectionSite`	string	Point 4 — North Shore
`collectionDate`	int (ms)	1714588800000
`dataType`	enum	absorbance
`notes`	string	free text
`isVerified`	bool	true
`verifiedBy`	string	jsilva
`result`	nested object	polymer, confidence, attentionMap, ...

Where each path is implemented

PATH PREFIX	DART SERVICE
`/institutions`	`lib/services/institution_service.dart`
`/institutions/<slug>/users`	`lib/services/auth_service.dart`
`/institutions/<slug>/datasets`	`lib/services/dataset_service.dart`
`/institutions/<slug>/samples`	`lib/services/sample_service.dart`

Identifier conventions

- `slug`: ASCII lowercase, hyphen-separated. Generated automatically by `InstitutionModel.slugify(name)`.
- `datasetId`: `ds-<timestamp>`, generated by `DatasetService.save`.
- `sampleId`: `smp-<12 random chars>`, generated by `SampleIdGenerator.generateInternalId()`.
- `sample.name`: `<prefix>-<6 random chars>`, e.g. `PF-K7XN3F`. The prefix comes from the dataset name (first letters of the first significant words).

SECTION 08

Flutter client architecture

There is no third-party state-management library: the app relies on `StatefulWidget` plus a single global `ChangeNotifier` for the locale. Each screen owns its own state and calls services directly. This keeps the navigation predictable and easy to reason about.

Layered structure

- `models/` — plain data classes with `toMap` / `fromMap` converters. No framework imports.
- `services/` — every HTTP call lives here. Returns models or null/booleans on failure. UI never imports `http` directly.
- `screens/` — one widget per screen. Reads from services in `initState`, refreshes on `pop`.
- `widgets/` — reusable visual components (`FtirChart`, `LanguageMenu`).
- `utils/` — pure functions (id generator).

Why no Provider / Riverpod / Bloc?

The app does not yet have a workflow that needs cross-screen reactive state. Every navigation is push/pop; data is reloaded from Firebase on return so the UI never goes stale. If the user base grows, switching to Riverpod is the cleanest upgrade path.

Custom-painted spectrum chart

`lib/widgets/ftir_chart.dart` draws the FTIR plot using `CustomPainter`. It overlays three layers: the intensity curve, the model's attention heatmap, and the decision-point dashed line. A toggle lets the user switch between absorbance and transmittance — derived on the fly via the `SpectrumSample.asTransmittance` / `asAbsorbance` getters.

SECTION 09

User flows

Each flow below is a vertical sequence of steps; arrows mark the natural progression a user follows. These are the four scenarios you will reproduce most often during development.

Cold start (database empty)

1

Login screen detects no institutions

On first boot the login screen has no slug to look up, so it shows a banner inviting the first administrator to register.

↓

2

Banner: *Create first admin*

Tapping it routes to the New Institution screen — the only place where a tenant can be created without already being signed in.

↓

3

Fill institution name + admin credentials

The slug is derived automatically from the name. Admin username becomes the first user inside the new tenant.

↓

4

Returns to login pre-filled with the slug

Both the institution and the admin are saved to Firebase in one atomic call; you only have to type your password.

↓

5

Sign in

Lands on the FTIR Overview (home) as the new admin.

Returning user

1. **Type the institution slug**

   Slug is the URL-safe identifier you chose when creating the tenant — e.g. `usp-iag`.

2. **Continue → app fetches /institutions/<slug>**

   If the slug exists, the screen swaps to ask for username and password; otherwise it shows a friendly error.

3. **Sign in**

   Validates the password hash and routes to the home screen.

Adding samples — three ways

1. **Open a collection**

   Home → tap a collection card → Dataset Detail screen.

2. **Tap “+” New Sample**

   Opens the registration form with two tabs: **Individual** and **Batch**.

3. **Tab “Individual”**

   Fill site / date / notes. Optionally **attach a single-row CSV** — it is sent to the MLP for instant identification before saving.

4. **Tab “Batch” → quantity**

   Saves N empty samples with random IDs (good for cataloguing field tubes before measuring them).

5. **Tab “Batch” → import from CSV**

   Routes to the Import CSV screen with site / notes inherited from the form. Bulk MLP inference, then mass save.

Running identification later

1. **Open a sample**

   Dataset Detail → tap a row to see its spectrum.

2. **Tap the biotech icon**

   Top-right of the Sample Detail screen. Triggers POST /predict on the MLP server.

3. **Server returns class, confidence, attentionMap**

   All three are persisted to Firebase under `samples/<id>/result`.

4. **Chart updates with attention heatmap**

   The orange overlay highlights the wavenumbers that pushed the decision; a yellow dashed line marks the strongest peak.

SECTION 10

Internationalisation (i18n)

The app starts in English. Users can switch to Portuguese from the language menu, available both on the login screen and inside the home menu. The choice is persisted in `SharedPreferences` by `LocaleService`, which exposes a `ChangeNotifier` that `RaviApp` listens to — switching the locale rebuilds the entire widget tree without restarting.

Adding a new translatable string

```
# 1. Add the new key to BOTH ARB files
#    (lib/l10n/app_en.arb and lib/l10n/app_pt.arb)
"helloUser": "Hello, {name}",
"@helloUser": { "placeholders": { "name": {} } }

# 2. Regenerate Dart files
flutter gen-l10n

# 3. Use it in any widget
final l = AppLocalizations.of(context);
Text(l.helloUser('Daniel'))
```

ARB conventions

- Keys use camelCase, grouped by screen/feature (`homeKpiTotalSamples`, `addSampleSiteHint`).
- Always edit both `app_en.arb` and `app_pt.arb` — missing keys throw at runtime.
- Use `@key` metadata for placeholders, e.g. `"@datasetSamples": { "placeholders": { "n": { "type": "int" } } }`.
- Generated files (`app_localizations*.dart`) are committed for editor support but are regenerated on every `flutter gen-l10n` — never edit them by hand.

RAVI acronym

The acronym `RAVI` is fixed across locales. Only the expansion changes: in English it stands for `Recognition Automated Via Infrared`, in Portuguese `Reconhecimento Automatizado Via Infravermelho`.

SECTION 11

Common tasks

Add a new screen

```
// 1. Create lib/screens/my_screen.dart
class MyScreen extends StatelessWidget { ... }

// 2. Add localisations to BOTH ARB files and run:
//      flutter gen-l10n

// 3. Navigate from wherever:
Navigator.push(context, MaterialPageRoute(
    builder: (_) => MyScreen(loggedUser: widget.loggedUser),
));
```

Add a new server endpoint

```
# 1. Add the route in mlp_server.py
@app.post("/something/{institution_slug}")
def my_route(institution_slug: str, req: MyRequest):
    ...

# 2. Add the Dart wrapper in services/
class SomethingService {
  static Future<...> call({...}) async {
    final res = await http.post(...);
    ...
  }
}

# 3. Restart the Python server (Uvicorn auto-reload is OFF by default).
```

Add a new polymer class

```
# 1. Retrain MLP_best_model.h5 with the extra class.
# 2. Update CLASS_NAMES in mlp_server.py.
# 3. Add the matching enum value in lib/models/spectrum_model.dart
#    (PolymerType, label, fullName, color).
# 4. Update _mapPolymer() in csv_import_service.dart and mlp_service.dart.
# 5. Optionally: add diagnostic peaks in the same files.
```

Change the Firebase database URL

Edit the constant `_base` in `lib/services/firebase_service.dart`. This is the single point of change — every other service derives its path from there.

Change the MLP server URL

There are three references to `http://localhost:8000`: `MlpService`, `CsvImportService` and `SpectralStorageService`. When deploying outside the dev box, extract these into a shared constant.

SECTION 12

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
“Could not reach the MLP server”	mlp_server.py not running, or wrong host	Run `python mlp_server.py`; verify `/health` returns 200
Login: *Institution not found*	Database empty or wrong slug	Use the first-run flow, or check `/institutions` in Firebase
FTIR chart blank after restart	Spectrum CSV missing on the server	Check `spectra_data/<inst>/<ds>.csv` exists
CSV import fails 400	CSV without numeric column headers	Make sure column names are wavenumbers (e.g. `600.0`)
UI still in Portuguese after change	SharedPreferences cached or hot reload skipped	Restart the app; `LocaleService` re-reads on boot
flutter gen-l10n complains	Missing key in one of the ARBs	Diff `app_en.arb` and `app_pt.arb` — every key must exist in both

SECTION 13

Roadmap and known limitations

Short term

- Migrate authentication from the current base64 hash to Firebase Authentication. The Dart layer already isolates auth in `AuthService`, so the swap is local.
- Replace the hardcoded `http://localhost:8000` in the three services with a build-time configurable constant.
- Add per-collection ACLs so a researcher only sees collections they participate in.

Medium term

- Hook a retraining pipeline to the verified samples: every time an admin marks a batch as `isVerified`, append those rows to the training set.
- Migrate the dataset / sample listings to Firestore (paginated queries) once the per-tenant data grows past a few thousand rows.
- Move FTIR chart rendering to GPU (Skia paint primitives) for smoother scrolling on long spectra.

Open questions

- Should institutions support sub-tenants (departments)? Today all users of a slug share everything.
- Storage strategy for very large CSVs (>10 MB) — keep on disk or move to object storage?
- Whether to expose a RAVI public API (e.g. for other research groups to submit spectra for blind testing).

SECTION A

Glossary

TERM	MEANING
FTIR	Fourier-Transform InfraRed spectroscopy — the technique used to record the absorbance / transmittance signal.
MLP	Multilayer Perceptron — feed-forward neural network used here to classify the polymer.
Saliency / attention	Per-wavenumber importance derived from the model gradient, `|grad × x|`. Visualised as the orange heatmap.
Tenant / institution	An isolated namespace in the database. Identified by a slug.
Slug	ASCII-only, lowercase, hyphenated identifier derived from the institution name.
ARB	Application Resource Bundle — JSON-based file format used by Flutter for translations.
Decision wavenumber	The single wavenumber that the model uses as the strongest evidence for the predicted polymer class. Drawn as a yellow dashed line.